

用 C++ 模拟黑洞

Simulating Black Holes in C++

从结构体到 GPU 计算着色器, 用真实物理方程绘制时空弯曲

目录

Table of Contents

01

项目蓝图

结构体设计 / 极坐标系统 / 史瓦西半径

0

物理引擎

2

测地线概念 / 史瓦西度规 / 时空弯曲

0

核心算法

3

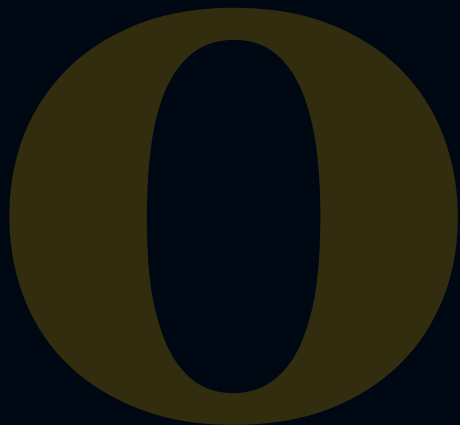
测地线方程 / RK4 积分器 / 算法对比

0

性能与可视化

4

3D 性能危机 / CPU 到 GPU / 吸积盘



项目蓝图

Project Blueprint

从 OpenGL 引擎到 struct 定义, 搭建 2D 光线追踪
的最小骨架

项目结构：两个 struct + 极坐标

BlackHole & Ray struct / Polar Coordinate System

BH

BlackHole struct

pos	Vec2	位置向量 (x, y)
mass	float	质量 M
rs	float	史瓦西半径

R

Ray struct

x, y	float	笛卡尔坐标
r, phi	float	极坐标
dr, dphi	float	极坐标速度

if (r < rs) skip step -> 阻止光线直接穿过黑洞

史瓦西半径：事件视界的几何

Schwarzschild Radius - the event horizon boundary

$$r_s = 2GM / c^2$$

Karl Schwarzschild - 1915

G

引力常数

6.674×10^{-11}
 $\text{N} \cdot \text{m}^2 / \text{kg}^2$

M

黑洞质量

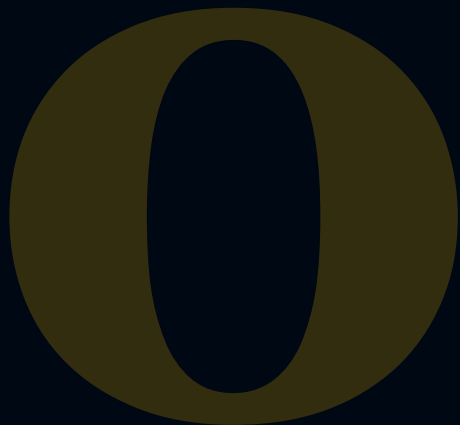
kg

c

光速

$2.998 \times 10^8 \text{ m/s}$

连光都无法逃逸的距离 = 事件视界 (Event Horizon)



物理引擎

Spacetime & Geodesics

测地线 / 史瓦西度规 / 爱因斯坦场方程 -- 引力 =
时空弯曲的几何效应

测地线：弯曲空间中的“直线”

Geodesics & Schwarzschild Metric

核心概念

- **测地线 (Geodesic)**

弯曲空间中的最短路径，平面地图上看似曲线

- **零测地线 (Null Geodesic)**

光走的特殊路径，光无时间维度

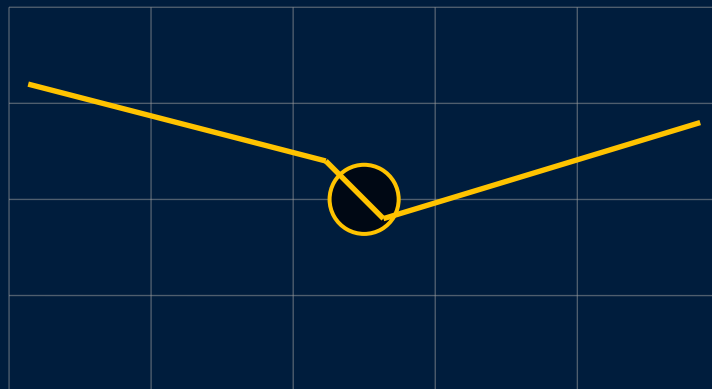
- **爱因斯坦场方程**

联系质量 - 能量与时空几何的核心方程

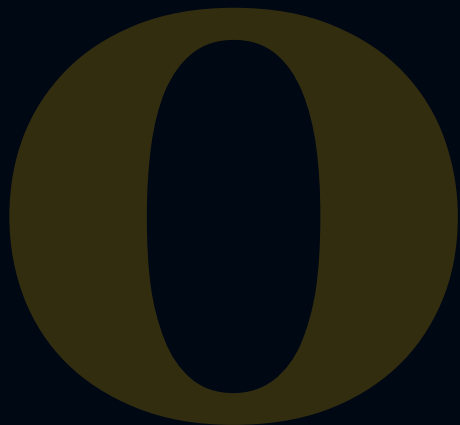
- **史瓦西度规**

1915 年求解非旋转黑洞的简化时空度规

时空网格 + 光线弯曲



平面地图上的弧线 = 球面上的最短路径



核心算法

Geodesic Equation & RK4 Integrator

从克里斯托费尔符号到四阶龙格 - 库塔积分, 精度决定真实性

测地线方程与克里斯托费尔符号

Geodesic Equation + Christoffel Symbols

测地线方程

$$d^2x / d\lambda^2 + \Gamma \cdot (dx/d\lambda)(dx/d\lambda) = 0$$

x

时空坐标

代表 $t/r/\phi$ 中的
任意一个

Γ

克里斯托费尔符号

编码黑洞引起的时空曲
率

λ

仿射参数

模拟中用于向前推进的
任意步长

带入 $x=\phi$ 化简得 $\Gamma=1/r$; 带入 $x=r$ 化简得 r^2 项 -- 下一张详细展开

两条核心方程 : r 与 ϕ 的加速度

Acceleration equations derived from Schwarzschild metric



角加速度

Angular acceleration

$$\frac{d^2\phi}{d\lambda^2} = -\left(\frac{2}{r}\right) \cdot \left(\frac{dr}{d\phi}\right)$$

光线相对于黑洞的角度变化率

-> 控制光线弯曲的方向



径向加速度

Radial acceleration

$$\frac{d^2r}{d\lambda^2} = -\left(\frac{c^2 \cdot r_s}{2r^2}\right) + r \cdot \left(\frac{d\phi}{d\lambda}\right)^2$$

光线靠近 / 远离黑洞的速度变化率

-> 控制光线被吸入的快慢

一阶导 -> 更新 $dr/d\phi$; 二阶导 -> 更新 r/ϕ 位置 -- 循环推进

积分器之争：欧拉法 vs 龙格 - 库塔四阶

Euler Method vs Runge-Kutta 4 (RK4)

BEFORE

欧拉法 (Euler)

$$v_{\text{new}} = v + a \cdot dt$$

+ 实现最简单，直接套公式

- 漆黑弯道高速行驶
- 大幅跳跃式计算
- 光线会 "跨步" 入黑洞
- 路径过直，丢失细节

AFTER

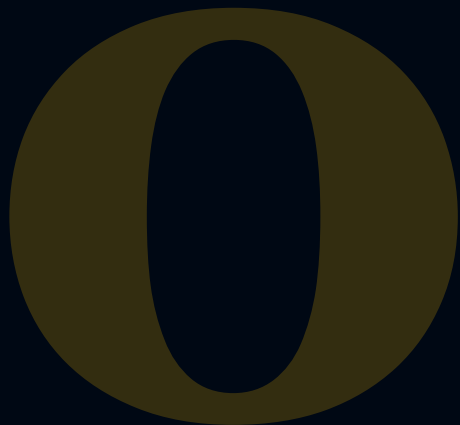
RK4

k1..k4 加权平均

+ 曲线更真实，无生硬跳跃

- 一次性 "走 4 步"
- 对 4 个状态加权求平均
- 精度提升 4 阶
- 接近 2D 模拟最高精度

结论: RK4 = 4 步加权平均, 精度远高于欧拉法



性能与可视化

Performance & Visualization

从 CPU 到 GPU 计算着色器, 以及最后的吸积盘与光环

3D 性能危机 :48 万条光线的算力账本

3D Performance Bottleneck at 800x600

800 x 600 x 20,000 x 4 = 帧计算量

屏幕像素 | 每像素光线数 | RK4 前推步数 | 每步 4 次测地线评估

480,000

光线 / 帧

800 x 600 屏幕

20,000

步 / 光线

RK4 前推距离

4x

评估 / 步

RK4 内部 4 步

0.5

FPS

CPU 几乎跑不动

结论：关闭弯曲后流畅，开启后卡顿 -> 瓶颈在计算，不在渲染

从 CPU 到 GPU: 麦当劳的启示

CPU vs GPU Compute Shader (Geodesic.comp)

BEFORE

CPU

CPU 串行

米其林星级餐厅

精致多样, 但每道菜单独做

- 48 万光线串行处理
- 每条光线 20,000 步 RK4
- 帧率接近 0.5 FPS
- 无法实时渲染

AFTER

GPU

GPU 并行

麦当劳流水线

单一品类, 快速可靠并行

- GLSL compute shader
- 光线并行求值, 无依赖
- 分辨率降至 400x300
- 实时流畅运行

Geodesic.comp - 2 万步循环内调用 RK4, 光线与黑洞相交则退出

关键点

Key Takeaways

01

时空弯曲 = 测地线

光线沿弯曲时空中的最短路径运动，而非传统力

02

史瓦西度规 = 非旋转黑洞解

求解爱因斯坦场方程的简化版，得到完整时空几何

03

RK4 优于欧拉法

4 步加权平均积分，精度决定光线的真实性

04

GPU 计算着色器解锁 3D

GLSL compute shader 并行处理，实现实时渲染



Thank You

原视频：youtube.com/watch?v=8-B6ryuBkCM

中配：bilibili.com/video/BV1VqshzdEKC